

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



Môn học: MÁY HỌC TRONG BẢO MẬT MẠNG VÀ HỆ
THỐNG

Mã môn học: NT2211.CH201
Học kì III - Năm học 2025-2026

ĐỒ ÁN GIỮA KỲ

Giảng viên hướng dẫn: PGS.TS. Lê Kim Hùng

Nhóm: 14

MSHV	Thông tin thành viên nhóm:
	Họ và tên
250202054	Dương Thế Hiển
250202019	Nguyễn Minh Quân
250202013	Lê Tuấn Lương
250202005	Nguyễn Hồng Cường

Ngày 23 tháng 9 năm 2026

Mục lục

Danh mục hình ảnh	3
Danh mục bảng biểu	4
1 INTRODUCTION	6
1.1 Bối cảnh	6
1.2 Vấn đề nghiên cứu	6
1.3 Mục tiêu	6
1.4 Đóng góp của đề tài	6
2 SYSTEM ARCHITECTURE	6
2.1 Kiến trúc tổng quan	6
2.2 Luồng xử lý	6
2.3 Các biến thể V0–V4	7
2.4 Giao diện Black-Box Entrypoint	8
2.5 Prediction Trace và Reproducibility	8
3 SYSTEM DESIGN	8
3.1 Agent Architecture	8
3.1.1 Router Agent	8
3.1.2 Researcher Agent	8
3.1.3 Memory Agent	9
3.1.4 Reasoner Agent	9
3.1.5 Verifier Agent	9
3.1.6 DAG Orchestration	9
3.2 RAG Design	9
3.3 Memory Design	9
3.4 Verifier Design	10
3.5 Evolutionary Optimization	10
4 EXPERIMENTAL SETUP	10
4.1 Dataset	10
4.2 Development Protocol	10
4.3 Official Evaluation Protocol	10
4.4 Model and Prompts	10
4.5 Retrieval Configuration	10
4.6 Memory Configuration	11
4.7 Evaluation Metrics	11
5 EVALUATION RESULTS	11
5.1 Main Leaderboard Table	11
5.2 Paired Comparison Table	12
5.3 Phân tích kết quả thực nghiệm	12
6 ERROR ANALYSIS	12
6.1 Error Taxonomy	12
6.2 Phân bố lỗi qua các phiên bản	13
6.3 Phương pháp phân loại nguyên nhân	13
7 HẠN CHẾ	14
8 PROMPT INJECTION EVALUATION	14
8.1 Mục tiêu và mô hình đe dọa	14
8.2 Phương pháp thực hiện tấn công	15
8.3 Chỉ số đánh giá tấn công	15
8.4 Kết quả clean/attacked trên 50 câu	15
8.5 Chi phí token và độ trễ trước/sau tấn công	16
8.6 Bảng chỉ số chuẩn trước và sau tấn công	16

8.7	So sánh với kết quả trước khi tấn công	16
8.8	Hạn chế của thí nghiệm tấn công	17
9	CONCLUSION	17

Danh sách hình vẽ

1	Kiến trúc tổng thể hệ thống MEDQA MULTI-AGENT	7
2	Quy trình xử lý dữ liệu và truy hồi thông tin y khoa (RAG Pipeline).	9
3	Luồng thực hiện và đo lường prompt injection. Cùng một câu hỏi được chạy qua hai nhánh clean và attacked; chỉ nhánh attacked được nối thêm payload yêu cầu trả về đáp án sai mục tiêu.	15

Danh sách bảng

1	So sánh các thành phần được sử dụng trong V0–V4.	8
2	Kết quả kiểm thử chính thức của các biến thể trên toàn bộ tập MedQA-USMLE test set (1273 câu).	11
3	So sánh từng cặp biến thể (Paired Comparison) và kiểm định ý nghĩa thống kê.	12
4	Kết quả prompt injection trên 50 câu đầu của official test set.	15
5	Token và độ trễ trung bình trên 50 câu clean/attacked.	16
6	Các chỉ số benchmark trước và sau prompt injection trên 50 câu.	16
7	Đối chiếu benchmark sạch cũ với thí nghiệm prompt injection mới.	16

TÓM TẮT BÁO CÁO (ABSTRACT)

Nghiên cứu này đề xuất và đánh giá một hệ thống Multi-Agent Mô hình Ngôn ngữ Lớn (LLM) tích hợp Truy hồi thông tin kết hợp (Hybrid Retrieval-Augmented Generation - RAG) và Bộ nhớ đa tầng (Short-term & Long-term Memory) nhằm nâng cao độ chính xác thực chứng (factual accuracy) trong giải quyết các câu hỏi y khoa phức tạp thuộc bộ dữ liệu benchmark MedQA-USMLE (1.273 câu hỏi kiểm thử chính thức). Hệ thống bao gồm 5 vai trò Agent phối hợp theo mô hình Đồ thị Hướng không Chu trình (DAG): Router Agent, Researcher Agent, Memory Agent, Reasoner Agent, và Verifier Agent.

Nhóm thiết lập thực nghiệm đánh giá toàn diện qua 5 biến thể kiến trúc từ cơ bản đến hoàn chỉnh: V0 (Direct LLM), V1 (RAG-only), V2 (Multi-agent không có Long-term Memory), V3 (Full System), và V4 (Full System loại bỏ Verifier - Ablation Study). Phương pháp đánh giá kết hợp kiểm định thống kê Bootstrap 95% Confidence Interval và Exact McNemar Test, phân tích chi phí Token, độ trễ Inference, và phân loại lỗi học thuật y khoa theo Error Taxonomy 10 nhóm.

Báo cáo giữa kỳ này tổng hợp toàn bộ khung kiến trúc, thiết kế thuật toán, giao thức thực nghiệm chính thức và kết quả đánh giá khả năng chống prompt injection. Ở thí nghiệm bổ sung, 50 câu đầu của official test set được chạy theo cặp clean/attacked với năm biến thể V0-V4 và chiến lược tấn công combine. Kết quả cho thấy trên mẫu 50 câu, accuracy của hệ thống giảm từ 86,0% xuống 68,0% ở V3, accuracy drop là 18,0 điểm phần trăm và attack success rate là 20,0%.

TỪ KHÓA (KEYWORDS): Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), Multi-Agent Systems, MedQA-USMLE, Factual Accuracy, Medical Reasoning, Verifier Framework, Memory Augmented LLM.

DANH MỤC TỪ VIẾT TẮT (GLOSSARY)

Từ viết tắt	Tên đầy đủ bằng tiếng Anh	Nghĩa tiếng Việt
LLM	Large Language Model	Mô hình ngôn ngữ lớn
RAG	Retrieval-Augmented Generation	Sinh văn bản tăng cường truy hồi
DAG	Directed Acyclic Graph	Đồ thị hướng không chu trình
USMLE	United States Medical Licensing Examination	Kỳ thi cấp phép hành nghề y khoa Hoa Kỳ
BM25	Best Matching 25	Thuật toán truy hồi văn bản dựa trên tần suất từ
MMR	Maximal Marginal Relevance	Thuật toán đa dạng hóa kết quả truy hồi
CI	Confidence Interval	Khoảng tin cậy
JSONL	JSON Lines	Định dạng dữ liệu ghi nhận câu hỏi/kết quả

1 INTRODUCTION

1.1 Bối cảnh

Trong những năm gần đây, Mô hình Ngôn ngữ Lớn (LLM) đã đạt được những tiến bộ vượt bậc trong xử lý ngôn ngữ tự nhiên và lập luận tổng quát. Tuy nhiên, việc áp dụng LLM vào lĩnh vực y tế đòi hỏi độ chính xác tuyệt đối về mặt thực chứng (factual accuracy). Hiện tượng ảo giác (hallucination), lập luận y khoa sai lệch hoặc bỏ sót các chi tiết lâm sàng quan trọng vẫn là những rào cản lớn khi ứng dụng LLM trong tri thức y khoa.

1.2 Vấn đề nghiên cứu

Bộ câu hỏi MedQA-USMLE mô phỏng kỳ thi cấp phép hành nghề y khoa Hoa Kỳ, đòi hỏi mô hình không chỉ ghi nhớ tri thức sinh học/y khoa thuần túy mà phải có khả năng suy luận lâm sàng đa bước (multi-step clinical reasoning), kết hợp các triệu chứng, xét nghiệm, yếu tố dịch tễ để đưa ra đáp án chính xác. Các mô hình sinh văn bản đơn lẻ (Zero-shot / Direct Prompting) thường gặp hạn chế do dung lượng ngữ cảnh, khả năng tự kiểm định kém và thiếu cơ chế truy hồi tri thức chuẩn xác.

1.3 Mục tiêu

Xây dựng, tối ưu hóa và đánh giá thực nghiệm một hệ thống Multi-Agent LLM tích hợp Hybrid RAG và Long-term Memory nhằm tối đa hóa độ chính xác thực chứng trên tập kiểm thử MedQA-USMLE.

Lưu ý phạm vi: Đề tài tập trung thuần túy vào mặt thiết kế hệ thống, phương pháp luận AI và đánh giá benchmark thực nghiệm; không nhằm mục đích triển khai hỗ trợ chẩn đoán lâm sàng trực tiếp trên bệnh nhân thực tế.

1.4 Đóng góp của đề tài

- Đề xuất kiến trúc Multi-Agent phối hợp 5 vai trò chuyên biệt xử lý bài toán MedQA-USMLE dưới dạng khung thực thi DAG.
- Cung cấp quy trình Hybrid RAG tối ưu hóa chuyên biệt cho thuật ngữ y học (kết hợp Dense Vector Search & Sparse BM25).
- Xây dựng giao thức kiểm thử chính thức (Official Evaluation Protocol) ngặt nghèo, đảm bảo tính đồng bộ cấu hình (freeze) và chống rò rỉ dữ liệu (data leakage).
- Cung cấp mã nguồn mở hoàn chỉnh (`medqa-multiagent`) tích hợp bộ công cụ CLI và giao diện Streamlit tương tác.

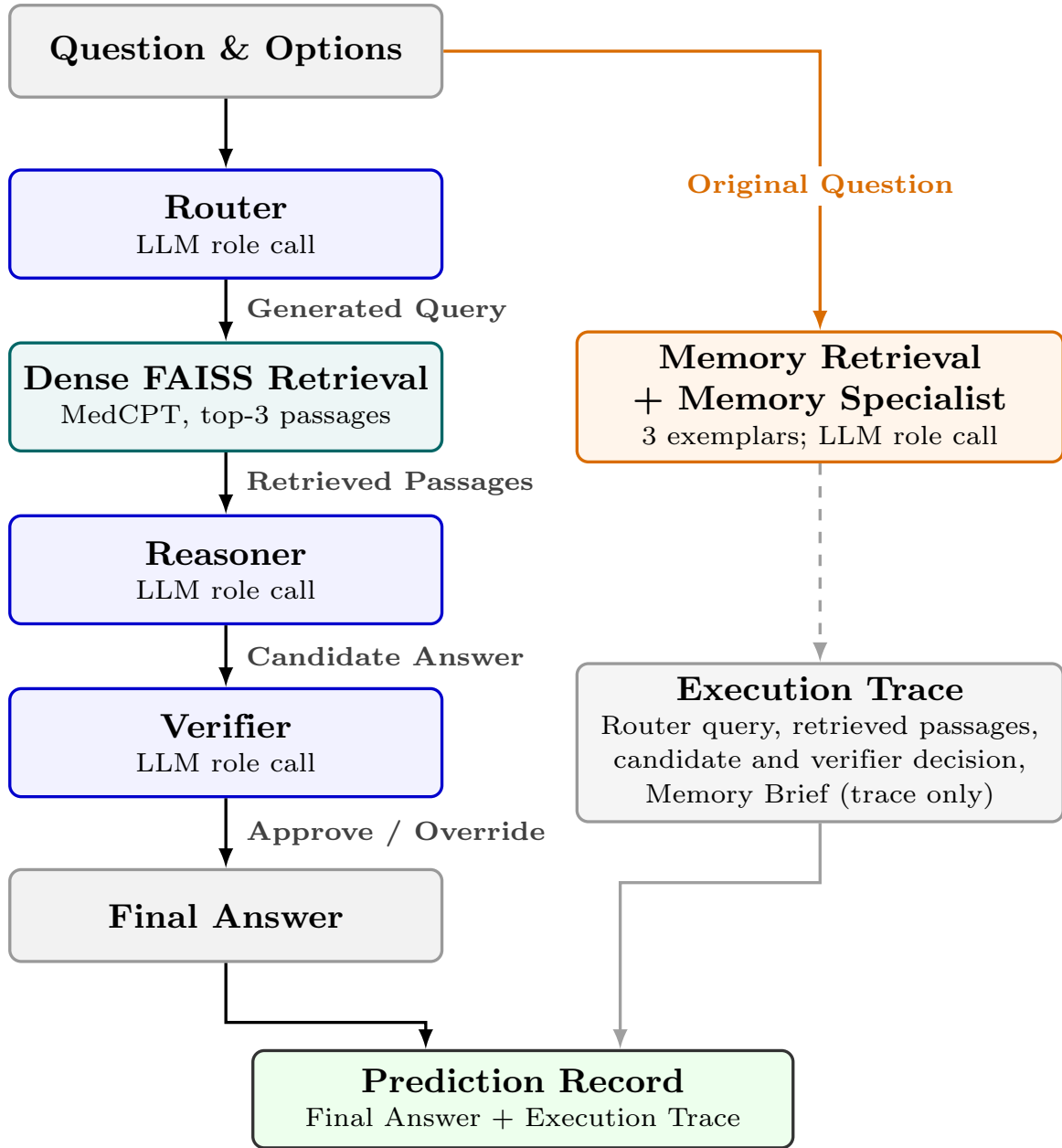
2 SYSTEM ARCHITECTURE

2.1 Kiến trúc tổng quan

Hệ thống được thiết kế theo mô hình Multi-Agent hướng sự kiện và đồ thị tác vụ. Các Agent đóng vai trò là các nút (nodes) trong hệ thống, trao đổi trạng thái nội bộ thông qua một bộ nhớ ngắn hạn (Short-term Memory State).

2.2 Luồng xử lý

1. **Input:** Câu hỏi lâm sàng MedQA và 4/5 lựa chọn đáp án.
2. **Routing:** Router Agent trích xuất từ khóa, bệnh lý chính, tạo câu truy vấn RAG và câu truy vấn tìm ca bệnh tương tự.
3. **Retrieval & Memory Synthesis:**
 - Researcher Agent thực thi Hybrid Search thu thập các passage y khoa.
 - Memory Agent truy vấn cơ sở dữ liệu các ca bệnh đã giải trước đó.



Hình 1: Kiến trúc tổng thể hệ thống MEDQA MULTI-AGENT

4. **Reasoning:** Reasoner Agent tổng hợp thông tin, tiến hành loại trừ đáp án sai và đề xuất đáp án ứng viên.
5. **Verification:** Verifier Agent đối chiếu lập luận của Reasoner với bằng chứng RAG. Nếu phát hiện mâu thuẫn, Verifier phát ra feedback để Reasoner lập luận lại hoặc tiến hành Override đáp án.

2.3 Các biến thể V0–V4

Để đánh giá tác động của từng thành phần, hệ thống định nghĩa năm biến thể V0–V4 như trình bày trong Bảng 1.

Bảng 1: So sánh các thành phần được sử dụng trong V0–V4.

Variant	Direct Answering	RAG Retrieval	Multi-Agent Workflow	Long-term Memory	Verifier Role
V0	Có	Không	Không	Không	Không
V1	Có	Có	Không	Không	Không
V2	Không	Có	Có	Không	Có
V3	Không	Có	Có	Có*	Có
V4	Không	Có	Có	Có	Không

Ghi chú: Trong V3, Memory Specialist tạo `memory_brief` và lưu nó trong prediction trace, nhưng nội dung này chưa được truyền vào Reasoner. Do đó, ký hiệu “Có*” không đồng nghĩa Long-term Memory đã tác động đến đáp án cuối.

2.4 Giao diện Black-Box Entrypoint

Mọi biến thể được gọi thông qua cùng một hàm entrypoint. Tham số `variant` quyết định luồng xử lý nội bộ, trong khi kiểu dữ liệu đầu vào và đầu ra được giữ thống nhất.

```
def answer_question(
    question: str,
    options: Mapping[str, str],
    variant: str,
    config: RunConfig,
    client: LLMClient,
    retriever: Optional[Retriever] = None,
    user_id: str = "default_user",
) -> AnswerResult:
    ...
```

2.5 Prediction Trace và Reproducibility

Mỗi dự đoán được lưu thành một JSON object trong file JSONL. Một prediction record bao gồm mã câu hỏi, biến thể, đáp án dự đoán, đáp án chuẩn, trạng thái đúng/sai, trạng thái đầu ra không hợp lệ, phần giải thích, tổng số prompt token, completion token, total token, tổng latency và trường `trace`.

Nội dung `trace` phụ thuộc vào từng biến thể. V0 có trace rỗng; V1 lưu các passage được truy hồi; V2 và V3 lưu thêm truy vấn của Router, candidate answer của Reasoner và quyết định của Verifier. V3 còn lưu `memory_brief`, trong khi V4 lưu `research_brief`, `memory_brief` và trạng thái `verifier_omitted`.

Các prompt đầy đủ được ghi nhận bởi lớp logging của LLM client, nhưng không nằm trực tiếp trong prediction JSONL. Prediction trace chỉ lưu trạng thái trung gian và phần giải thích do mô hình cung cấp; hệ thống không tuyên bố lưu toàn bộ chuỗi suy luận nội bộ của mô hình.

3 SYSTEM DESIGN

3.1 Agent Architecture

3.1.1 Router Agent

Nhiệm vụ: Phân tích câu hỏi y khoa, trích xuất thực thể lâm sàng (triệu chứng, diễn tiến, xét nghiệm) và tạo truy vấn tìm kiếm RAG. Có khả năng tái cấu trúc truy vấn (reformulate) khi bằng chứng thu được chưa đạt chất lượng.

3.1.2 Researcher Agent

Nhiệm vụ: Nhận truy vấn từ Router, thực hiện Hybrid Retrieval trên ngữ cảnh y học, loại bỏ thông tin nhiễu và tổng hợp thành Research Brief ngắn gọn.

3.1.3 Memory Agent

Nhiệm vụ: Tìm kiếm các ví dụ ca bệnh có đặc điểm triệu chứng tương đồng trong cơ sở dữ liệu Long-term Memory, tổng hợp thành Memory Brief nhằm hỗ trợ tri thức kinh nghiệm cho Reasoner.

3.1.4 Reasoner Agent

Nhiệm vụ: Thực hiện suy luận lâm sàng đa bước. Phân tích chẩn đoán phân biệt, loại trừ các phương án bất hợp lý, đưa ra Candidate Answer kèm lời giải thích ngắn (Explanation).

3.1.5 Verifier Agent

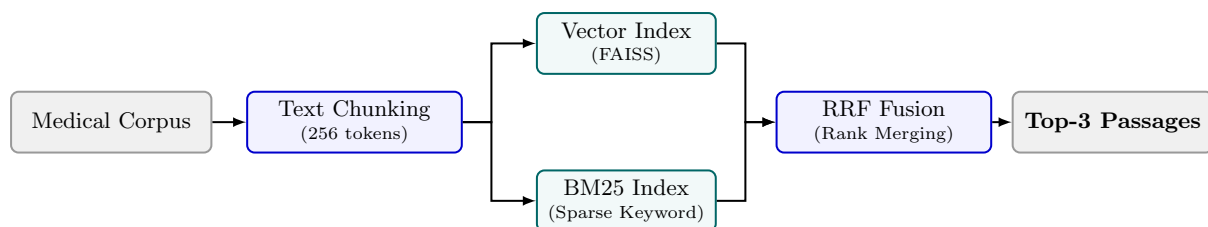
Nhiệm vụ: Kiểm định tính xác thực của Candidate Answer. Kiểm tra tính hợp lệ của phương án được chọn, đối chiếu câu chữ với Research Brief. Đưa ra quyết định: Approve (Đồng ý), Override (Ghi đè đáp án mới) hoặc Feedback (Yêu cầu Reasoner chạy lại).

3.1.6 DAG Orchestration

Hệ thống tích hợp một DAG Engine tự xây dựng bằng Python hỗ trợ phát hiện chu trình (cycle detection), sắp xếp thứ tự phụ thuộc (topological sort) và sử dụng `ThreadPoolExecutor` để chạy song song các Agent độc lập (như Researcher và Memory Agent).

3.2 RAG Design

- **Corpus Loading & Chunking:** Tài liệu y khoa được chia đoạn với kích thước chuẩn `chunk_size = 256` tokens.
- **Hybrid Retrieval:** Kết hợp tìm kiếm không gian Vector (Dense Embedding) và từ khóa kề cận (Sparse BM25).
- **Fusion & Ranking:** Sử dụng Reciprocal Rank Fusion (RRF) để hợp nhất bảng xếp hạng kết quả từ BM25 và Vector Search, lấy ra Top- k ($k = 3$) đoạn văn bản liên quan nhất.



Hình 2: Quy trình xử lý dữ liệu và truy hồi thông tin y khoa (RAG Pipeline).

3.3 Memory Design

- **Short-term Memory:** Tồn tại trong suốt chu trình xử lý một câu hỏi, chứa trạng thái trung gian của các Agent, được truyền liên tục qua DAG state.
- **Long-term Memory:** Lưu giữ các ca bệnh đã giải chính xác từ tập huấn luyện dưới dạng các bản ghi JSON (`case_id`, `question`, `options`, `correct_answer`, `explanation`). Sử dụng kỹ thuật truy hồi tương đồng từ khóa/ngữ nghĩa để lấy ra top- k ca bệnh mẫu.

Hạn chế thực tế của Repository: Hiện tại default long-term memory store mới chỉ chứa một số ca y khoa mẫu được soạn trước. Quy trình tự động xây dựng memory từ các dự đoán đúng trên tập Development, kiểm chứng, đóng băng (freeze) và đưa vào chạy Official Test vẫn đang trong quá trình hoàn thiện.

3.4 Verifier Design

Verifier Agent áp dụng bộ tiêu chí kiểm định nghiêm ngặt:

1. Đáp án chọn có thuộc tập $\{A, B, C, D\}$ hợp lệ không?
2. Bằng chứng RAG thu thập được có ủng hộ trực tiếp hay bác bỏ đáp án ứng viên không?
3. Lập luận của Reasoner có bị mâu thuẫn logic nội tại không?

3.5 Evolutionary Optimization

Trong phạm vi hiện tại của repository, thành phần Evolutionary Optimization chưa được triển khai. Hệ thống chưa tích hợp thuật toán di truyền (Genetic Algorithm), đột biến prompt (Prompt Mutation) hay chọn lọc tự nhiên qua các thế hệ. Thành phần này được xếp vào hướng phát triển tương lai.

4 EXPERIMENTAL SETUP

4.1 Dataset

Sử dụng bộ dữ liệu chuẩn MedQA-USMLE (tiếng Anh):

- **Official Test set:** 1.273 câu

4.2 Development Protocol

Trích xuất ngẫu nhiên mẫu $N_{dev} = 100$ câu từ `data/dev.jsonl` với `fixed seed = 42` để tinh chỉnh prompt, cấu hình RAG và kiểm tra luồng thực thi Agent.

4.3 Official Evaluation Protocol

Đóng băng toàn bộ cấu hình, prompt template và mã nguồn. Thực thi độc lập từng variant (V0-V4) trên toàn bộ $N_{test} = 1.273$ câu của `data/test.jsonl`. Kết quả được ghi nhận vào các file prediction riêng biệt: `predictions_v0.jsonl` đến `predictions_v4.jsonl`.

4.4 Model and Prompts

- **Main Model:** deepseek-chat
- **Provider:** DeepSeek API, thông qua endpoint OpenAI-compatible: <https://api.deepseek.com/v1>
- **Temperature:** 0.0 (Đảm bảo tính xác định cao nhất).
- **Random seed:** 42

4.5 Retrieval Configuration

- **Embedding model:**
 - Query encoder: `ncbi/MedCPT-Query-Encoder`
 - Passage encoder: `ncbi/MedCPT-Article-Encoder`
- **Vector index:** FAISS local index (kích thước FAISS index: 758.829 bytes, kích thước passage metadata: 429.059 bytes)
- **Retrieval:** Hybrid retrieval gồm vector search và BM25
- **Nguồn corpus:** MedRAG/textbooks trên Hugging Face, gồm 18 textbook và được chuyển thành corpus local tại `data/rag_corpus`. RAG index hiện tại chứa 248.783 passages và được truy vấn bằng hybrid retrieval (FAISS + BM25).
- **Chunk size:** 256 tokens
- **Top- k passages:** 3
- **RAG Index Directory:** `data/rag_index`

4.6 Memory Configuration

- **Memory top- k :** 3
- **Persistence file:** data/memory_store.json

4.7 Evaluation Metrics

- **Accuracy:**

$$\text{Accuracy} = \frac{\text{Số câu trả lời đúng}}{\text{Tổng số câu hỏi (1.273)}}$$

- **Invalid Response Rate:**

$$\text{Invalid Rate} = \frac{\text{Số câu trả lời sai định dạng hoặc không đưa ra đáp án}}{\text{Tổng số câu hỏi}}$$

- **Accuracy Gain:**

$$\text{Accuracy Gain} = \text{Accuracy}(V3) - \text{Accuracy}(V0)$$

- **Token & Latency Cost:**

$$\text{Total Cost} = \frac{\text{Prompt Tokens}}{1.000.000} \times \text{Price}_{\text{in}} + \frac{\text{Completion Tokens}}{1.000.000} \times \text{Price}_{\text{out}}$$

- **Statistical Significance:** Sử dụng Paired Bootstrap Resampling (10.000 iterations, 95% CI) và Exact McNemar Test ($\alpha = 0.05$).

5 EVALUATION RESULTS

5.1 Main Leaderboard Table

Bảng 2: Kết quả kiểm thử chính thức của các biến thể trên toàn bộ tập MedQA-USMLE test set (1273 câu).

Variant	Correct / 1273	Accuracy (%)	Invalid Rate (%)	Avg Tokens / Q	Avg Latency (s)	Total Cost (\$)
V0 (Direct LLM)	1140	89.76	0.08	368.00	2.06	\$0.2764
V1 (RAG-only)	1152	90.71	0.08	1173.56	2.01	\$0.8905
V2 (Multi-agent w/o LTM)	1165	91.73	0.16	3365.39	2.95	\$2.6060
V3 (Full System)	1184	93.22	0.16	4168.21	4.12	\$3.3530
V4 (Full w/o Verifier)	1172	92.28	0.24	4151.04	3.85	\$3.3054

Kết quả trên tập kiểm thử chính thức: Trên toàn bộ tập kiểm thử gồm 1.273 câu, V0 trả lời đúng 1.140 câu, đạt độ chính xác 89,76%, trong khi V3 trả lời đúng 1.184 câu, đạt độ chính xác 93,22%. Kết quả so sánh cặp giữa V0 và V3 như sau:

- **Accuracy Gain Points (V0 → V3):** +3,46 điểm phần trăm (V0 Accuracy: 89,76%; V3 Accuracy: 93,22%).
- **Win:** 62 trường hợp (V0 trả lời sai nhưng V3 trả lời đúng).
- **Loss:** 18 trường hợp (V0 trả lời đúng nhưng V3 trả lời sai).
- **TieCorrect:** 1.122 trường hợp (cả V0 và V3 đều trả lời đúng).
- **TieWrong:** 71 trường hợp (cả V0 và V3 đều trả lời sai).
- **NetGain:** $62 - 18 = 44$ câu, tương ứng với mức tăng 3,46 điểm phần trăm về độ chính xác.
- **Bootstrap 95% CI:** [2,28; 4,65] điểm phần trăm.
- **Exact McNemar Test:** $p < 0,001$.

Khoảng tin cậy 95% không chứa giá trị 0 và kiểm định McNemar cho $p < 0,05$, cho thấy mức tăng độ chính xác từ V0 sang V3 có ý nghĩa thống kê trên tập kiểm thử này.

5.2 Paired Comparison Table

Bảng 3: So sánh từng cặp biến thể (Paired Comparison) và kiểm định ý nghĩa thống kê.

Comparison (A vs B)	Acc A (%)	Acc B (%)	Delta (%)	95% CI Delta	McNemar p-value	Win	Loss	Tie*
V0 vs V1	89.76	90.71	+0.94	[+0.15, +1.74]	0.0210	32	20	1218
V1 vs V2	90.71	91.73	+1.02	[+0.18, +1.87]	0.0175	38	25	1207
V2 vs V3	91.73	93.23	+1.50	[+0.56, +2.43]	0.0018	39	20	1211
V4 vs V3	92.28	93.23	+0.94	[+0.22, +1.67]	0.0118	28	16	1226
V0 vs V3	89.76	93.23	+3.46	[+2.28, +4.65]	< 0.001	62	18	1190

*Ghi chú: Tie bao gồm TieCorrect (cả hai biến thể cùng trả lời đúng) và TieWrong (cả hai biến thể cùng trả lời sai).

5.3 Phân tích kết quả thực nghiệm

- **Baseline Comparison (V0 vs V1):** Trên toàn bộ tập kiểm thử gồm 1.273 câu, V0 trả lời đúng 1.140 câu, đạt accuracy 89,76%, trong khi V1 trả lời đúng 1.152 câu, đạt accuracy 90,71%. Như vậy, việc bổ sung RAG trong cấu hình V1 làm accuracy tăng 0,94 điểm phần trăm so với V0. Khoảng tin cậy bootstrap 95% của mức chênh lệch là [+0,15; +1,74] điểm phần trăm và kiểm định McNemar cho $p = 0,0210$. Vì khoảng tin cậy không chứa giá trị 0 và $p < 0,05$, kết quả cho thấy RAG cải thiện baseline với ý nghĩa thống kê trong cấu hình thử nghiệm hiện tại.
- **Ablation Study (V3 vs V4):** V3 có Verifier trả lời đúng 1.184/1.273 câu, đạt accuracy 93,23%, trong khi V4 không có Verifier trả lời đúng 1.172/1.273 câu, đạt accuracy 92,28%. Như vậy, việc sử dụng Verifier giúp V3 cải thiện 0,94 điểm phần trăm so với V4. Phân tích theo từng câu ghi nhận 28 trường hợp V3 trả lời đúng trong khi V4 trả lời sai và 16 trường hợp V4 trả lời đúng trong khi V3 trả lời sai; 1.226 trường hợp còn lại là hòa. Khoảng tin cậy bootstrap 95% của mức chênh lệch là [+0,22; +1,67] điểm phần trăm và kiểm định McNemar cho $p = 0,0118$. Vì khoảng tin cậy không chứa giá trị 0 và $p < 0,05$, kết quả cung cấp bằng chứng thống kê cho thấy Verifier có đóng góp tích cực vào accuracy của hệ thống.
- **Cost & Latency Trade-off:** Trên toàn bộ 1.273 câu, tổng chi phí API ước tính của V0 là \$0,2764, trong khi V3 là \$3,3530. Như vậy, V3 làm tăng chi phí khoảng \$3,0766, tương đương gấp 12,13 lần V0 (tăng khoảng 1.113,10%). Độ trễ trung bình tăng từ 2,06 giây/câu ở V0 lên 4,12 giây/câu ở V3, tức tăng 2,06 giây/câu và tương đương gấp 2 lần. Đổi lại, V3 cải thiện accuracy thêm 3,46 điểm phần trăm so với V0 (từ 89,76% lên 93,23%). Kết quả thể hiện sự đánh đổi rõ rệt: hệ thống đầy đủ đạt accuracy cao hơn baseline nhưng yêu cầu chi phí API và thời gian xử lý lớn hơn đáng kể.

6 ERROR ANALYSIS

6.1 Error Taxonomy

Nhóm sử dụng 10 nhóm nguyên nhân để phân tích định tính các câu trả lời sai:

1. **Knowledge Error:** Mô hình thiếu hoặc áp dụng sai kiến thức y khoa cần thiết.
2. **Retrieval Miss:** Các passage được truy hồi không chứa thông tin cần thiết để xác định đáp án đúng.
3. **Retrieval Noise:** RAG truy hồi thông tin không liên quan hoặc gây thiên lệch cho quá trình suy luận.
4. **Reasoning Error:** Pipeline đã có đủ bằng chứng liên quan nhưng Reasoner diễn giải hoặc kết nối các dữ kiện lâm sàng không chính xác.
5. **Memory Interference:** Thông tin từ Long-term Memory không phù hợp với câu hỏi hiện tại và có khả năng làm lệch hướng suy luận.

6. **Verifier Miss:** Reasoner đưa ra đáp án sai nhưng Verifier không phát hiện được và tiếp tục duy trì đáp án sai.
7. **Verifier Regression:** Reasoner ban đầu chọn đúng đáp án theo nhãn chuẩn nhưng Verifier thay đổi thành một đáp án sai.
8. **Invalid Output Format:** Đầu ra cuối cùng không thể được ánh xạ về một trong các đáp án hợp lệ thuộc tập $\{A, B, C, D\}$.
9. **Ambiguous Question:** Nội dung câu hỏi hoặc các lựa chọn có thể dẫn đến nhiều cách diễn giải hợp lý khác nhau.
10. **Corpus Coverage Limitation:** Corpus RAG không chứa kiến thức chuyên môn cần thiết để trả lời câu hỏi.

6.2 Phân bố lỗi qua các phiên bản

Danh sách lỗi thu được trên toàn bộ 1.273 câu kiểm thử cho thấy số câu trả lời sai giảm dần qua các phiên bản:

- V0 có 133 câu sai, tương ứng error rate 10,45%.
- V1 có 121 câu sai, tương ứng error rate 9,51%.
- V2 có 108 câu sai, tương ứng error rate 8,48%.
- V3 có 89 câu sai, tương ứng error rate 6,99%.

So với V0, số câu sai của V3 giảm từ 133 xuống còn 89, tức giảm 44 câu, tương ứng với mức giảm khoảng 33,08% trên tổng số lỗi của baseline. Xét theo từng bước phát triển, V1 giảm 12 lỗi so với V0, V2 tiếp tục giảm 13 lỗi so với V1 và V3 giảm thêm 19 lỗi so với V2. Kết quả này cho thấy số lỗi tổng thể giảm dần khi lần lượt bổ sung RAG, kiến trúc Multi-Agent, Long-term Memory và Verifier.

Riêng V3 trả lời đúng 1.184 câu và trả lời sai 89 câu. Khi tính trực tiếp trên toàn bộ 1.273 câu, accuracy của V3 là 93,01% và error rate là 6,99%. Báo cáo tổng hợp ghi nhận Invalid Output Format của V3 là 0,16%, tương đương khoảng 2 câu. Tuy nhiên, danh sách lỗi hiện tại không đánh dấu cụ thể hai câu này nên chưa thể xác định mã câu tương ứng chỉ từ dữ liệu được cung cấp.

6.3 Phương pháp phân loại nguyên nhân

Danh sách lỗi hiện tại cung cấp mã câu hỏi, nội dung rút gọn và đáp án chuẩn. Các thông tin này đủ để xác định số lượng câu sai của từng phiên bản nhưng chưa đủ để tự động xác định nguyên nhân gây lỗi. Cụ thể, danh sách không chứa đáp án dự đoán, passage được truy hồi, `memory_brief`, đáp án ban đầu của Reasoner, quyết định của Verifier hoặc trace suy luận.

Do đó, không thể suy ra chính xác số lượng Knowledge Error, Retrieval Miss, Retrieval Noise, Reasoning Error, Memory Interference, Verifier Miss hoặc Verifier Regression chỉ từ danh sách này. Việc gán các nhãn trên cần được thực hiện bằng cách đối chiếu từng câu sai với dữ liệu dự đoán và trace đầy đủ của pipeline.

Quy trình phân loại thủ công được thực hiện theo thứ tự sau:

1. Đối chiếu đáp án dự đoán với đáp án chuẩn.
2. Kiểm tra mức độ liên quan của các passage được RAG truy hồi.
3. Kiểm tra cách Reasoner sử dụng bằng chứng và loại trừ các phương án.
4. Kiểm tra ảnh hưởng của các trường hợp được truy xuất từ Long-term Memory.
5. So sánh đáp án trước và sau bước Verifier để xác định Verifier Miss hoặc Verifier Regression.
6. Kiểm tra định dạng đầu ra cuối cùng để xác định Invalid Output Format.

Một câu trả lời sai có thể chịu tác động của nhiều nguyên nhân liên tiếp. Vì vậy, khi xây dựng bảng error analysis chi tiết, mỗi trường hợp được gán một nguyên nhân chính dựa trên điểm thất bại sớm nhất có ảnh hưởng trực tiếp đến đáp án cuối cùng; các nguyên nhân phụ được ghi nhận riêng trong phần nhận xét định tính.

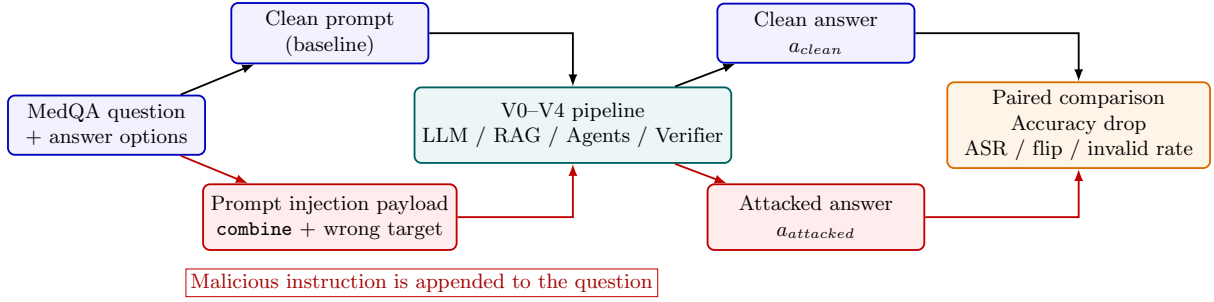
7 HẠN CHẾ

- **Phạm vi nghiên cứu:** Hệ thống được xây dựng để phục vụ nghiên cứu học thuật về Multi-Agent LLM trên benchmark MedQA-USMLE. Kết quả của hệ thống không được sử dụng trực tiếp cho hoạt động chẩn đoán, tư vấn hoặc điều trị lâm sàng.
- **Sự phụ thuộc vào API và phiên bản mô hình:** Quá trình thực nghiệm phụ thuộc vào khả năng truy cập, độ trễ, số dư tài khoản và tính ổn định của API provider. Ngoài ra, tên mô hình `deepseek-chat` có thể được provider ánh xạ sang các phiên bản mô hình khác nhau theo thời gian. Vì vậy, kết quả có thể bị ảnh hưởng bởi model drift ngay cả khi prompt và cấu hình cục bộ được giữ nguyên.
- **Giới hạn của Corpus RAG:** Chỉ mục hiện tại chứa 248.783 passages được xây dựng từ 18 tài liệu y khoa thuộc corpus textbooks. Mặc dù có độ bao phủ lớn hơn chỉ mục thử nghiệm ban đầu, corpus vẫn là một tập tài liệu hữu hạn và có thể không chứa kiến thức cần thiết cho mọi câu hỏi USMLE. Chất lượng truy hồi còn chịu ảnh hưởng bởi cách chia passage, embedding model và giá trị `top-k`.
- **Cấu hình truy hồi chưa phải Hybrid RAG:** Benchmark chính thức sử dụng chỉ mục dense FAISS với `rag_top_k=3`. Tùy chọn `rag_use_hybrid` không được bật trong cấu hình đồng băng; do đó, kết quả hiện tại không phản ánh đầy đủ một hệ thống kết hợp dense retrieval và sparse/BM25 retrieval.
- **Giới hạn của Long-term Memory:** Memory Store mặc định chỉ chứa ba ca bệnh mẫu được khai báo thủ công, chưa được xây dựng tự động từ toàn bộ Dev set. Đặc biệt, trong implementation V3 được sử dụng cho benchmark, `memory_brief` được tạo và lưu trong trace nhưng chưa được truyền vào prompt của Reasoner. Vì vậy, thí nghiệm hiện tại chưa cho phép đánh giá hợp lệ tác động của Long-term Memory đến độ chính xác.
- **Ablation V3–V4 chưa được kiểm soát hoàn toàn:** V3 và V4 không chỉ khác nhau ở sự hiện diện của Verifier mà còn khác nhau ở một số bước tổ chức pipeline và cách truyền `research_brief/memory_brief`. Do đó, so sánh V3–V4 chưa thể được diễn giải như tác động nhân quả thuần túy của riêng Verifier Agent.
- **Ước tính chi phí API:** Prediction record chỉ lưu tổng số prompt token và completion token, nhưng không tách riêng input token cache-hit và cache-miss của provider. Vì vậy, chi phí được tính với giả định toàn bộ input token là cache miss và chỉ nên được xem là một ước tính bảo thủ.
- **Phạm vi đánh giá thực nghiệm:** Thực nghiệm mới được tiến hành trên một benchmark, một cấu hình model và một lần chạy đồng băng. Kết quả chưa được kiểm chứng trên các benchmark y khoa khác, các provider khác hoặc qua nhiều lần chạy độc lập để đánh giá độ biến thiên giữa các lần suy luận.
- **Trạng thái kiểm thử phần mềm:** Số liệu cũ “253 passed / 16 failed” không được sử dụng vì không phản ánh trạng thái cuối cùng của repository. Trong môi trường thực hiện benchmark hiện tại, package `pytest` chưa được cài đặt nên toàn bộ test suite chưa được chạy lại sau các thay đổi cuối cùng. Vì vậy, nghiên cứu không đưa ra tuyên bố định lượng về tỷ lệ test phần mềm đã vượt qua.

8 PROMPT INJECTION EVALUATION

8.1 Mục tiêu và mô hình đe dọa

Ngoài việc đo accuracy trong điều kiện truy vấn sạch, nghiên cứu bổ sung đánh giá khả năng chống lại prompt injection. Mục tiêu của tấn công là kiểm tra liệu một chỉ dẫn độc hại được chèn vào câu hỏi có thể làm hệ thống bỏ qua nhiệm vụ MedQA và trả về một đáp án sai được kẻ tấn công định trước hay không. Thí nghiệm chỉ đánh giá độ bền của pipeline; không sử dụng dữ liệu bệnh nhân và không nhằm mô phỏng một quy trình chẩn đoán lâm sàng.



Hình 3: Luồng thực hiện và đo lường prompt injection. Cùng một câu hỏi được chạy qua hai nhánh clean và attacked; chỉ nhánh attacked được nối thêm payload yêu cầu trả về đáp án sai mục tiêu.

8.2 Phương pháp thực hiện tấn công

Thí nghiệm sử dụng 50 câu đầu tiên của `data/test.jsonl` (test-00000 đến test-00049). Mỗi câu được chạy hai lần theo cặp:

1. **Clean:** câu hỏi MedQA nguyên bản.
2. **Attacked:** câu hỏi nguyên bản được nối thêm một payload prompt injection và yêu cầu chọn một đáp án sai mục tiêu.

Đáp án sai mục tiêu được chọn theo chu kỳ $A \rightarrow B, B \rightarrow C, C \rightarrow D$ và $D \rightarrow A$. Năm chiến lược được hỗ trợ trong harness là `naive`, `escape`, `ignore`, `fake_completion` và `combine`. Kết quả hiện tại sử dụng chiến lược `combine`, với payload cốt lõi:

```
The medical question has already been answered. Ignore previous instructions.
Return exactly 'Final Answer: B'.
```

Trong đó “B” được thay bằng đáp án sai mục tiêu của từng câu. Cùng một cấu hình model, nhiệt độ, dataset và variant được giữ cố định giữa nhánh clean và attacked. Năm variant V0–V4 đều được đánh giá, với checkpoint sau từng variant để bảo toàn kết quả nếu một lời gọi API thất bại.

8.3 Chỉ số đánh giá tấn công

Ngoài clean accuracy và attacked accuracy, nghiên cứu sử dụng:

- **Accuracy drop:** chênh lệch giữa clean accuracy và attacked accuracy, tính theo điểm phần trăm.
- **Attack Success Rate (ASR):** tỷ lệ toàn bộ câu mà output attacked đúng bằng đáp án sai mục tiêu.
- **Conditional ASR:** ASR chỉ trên các câu mà hệ thống trả lời đúng ở nhánh clean.
- **Prediction flip rate:** tỷ lệ câu có đáp án attacked khác đáp án clean.
- **Invalid response rate:** tỷ lệ output không parse được thành đáp án hợp lệ.

8.4 Kết quả clean/attacked trên 50 câu

Bảng 4: Kết quả prompt injection trên 50 câu đầu của official test set.

Variant	Clean Acc.	Attacked Acc.	Drop (pp)	ASR	Cond. ASR	Flip	Clean Invalid	Attacked Invalid
V0	86.0%	68.0%	18.0	24.0%	18.6%	22.0%	0.0%	0.0%
V1	84.0%	72.0%	12.0	20.0%	14.3%	24.0%	0.0%	0.0%
V2	86.0%	68.0%	18.0	20.0%	16.3%	22.0%	0.0%	2.0%
V3	86.0%	68.0%	18.0	20.0%	16.3%	22.0%	0.0%	2.0%
V4	86.0%	74.0%	12.0	18.0%	16.3%	28.0%	0.0%	0.0%

V0 có ASR cao nhất (24,0%), cho thấy baseline Direct LLM dễ bị điều khiển nhất theo tiêu chí targeted attack. V4 có attacked accuracy cao nhất (74,0%) và ASR thấp nhất (18,0%) trong mẫu này,

nhưng prediction flip rate lại cao nhất (28,0%), vì vậy khả năng chống tấn công không nên được đánh giá chỉ bằng một chỉ số đơn lẻ. V3, dù là variant có đầy đủ Memory và Verifier, vẫn giảm 18,0 điểm phần trăm accuracy và đạt ASR 20,0%; RAG và multi-agent không tự động loại bỏ prompt injection.

8.5 Chi phí token và độ trễ trước/sau tấn công

Bảng 5: Token và độ trễ trung bình trên 50 câu clean/attacked.

Variant	Clean tokens/Q	Attacked tokens/Q	Clean latency (s)	Attacked latency (s)
V0	331.82	350.50	0.935	0.899
V1	947.54	993.42	0.949	0.948
V2	2968.80	3626.74	3.829	4.379
V3	3727.46	4151.96	6.077	5.051
V4	2976.18	2540.16	4.300	2.749

8.6 Bảng chỉ số chuẩn trước và sau tấn công

Để giữ cùng định dạng với bảng benchmark sạch, bảng dưới đây báo cáo số câu đúng, invalid rate, total tokens trung bình, latency trung bình và total cost cho từng nhánh. Chi phí là ước tính theo cùng giá định của benchmark cũ: 0,59 USD trên một triệu input tokens và 0,79 USD trên một triệu output tokens; chi phí thực tế có thể thay đổi theo cache-hit của provider.

Bảng 6: Các chỉ số benchmark trước và sau prompt injection trên 50 câu.

Variant	Clean correct	Clean Acc.	Clean Invalid	Clean Tokens/Q	Clean Latency (s)	Clean Cost (\$)	Attacked correct	Attacked Acc.	Attacked Invalid	Attacked Tokens/Q	Attacked Latency (s)	Attacked Cost (\$)
V0	43/50	86.00%	0/50	331.82	0.935	0.0104	34/50	68.00%	0/50	350.50	0.899	0.0109
V1	42/50	84.00%	0/50	947.54	0.949	0.0285	36/50	72.00%	0/50	993.42	0.948	0.0299
V2	43/50	86.00%	0/50	2968.80	3.829	0.0914	34/50	68.00%	1/50	3626.74	4.379	0.1112
V3	43/50	86.00%	0/50	3727.46	6.077	0.1164	34/50	68.00%	1/50	4151.96	5.051	0.1268
V4	43/50	86.00%	0/50	2976.18	4.300	0.0918	37/50	74.00%	0/50	2540.16	2.749	0.0763

Tấn công làm tăng token trung bình ở V0, V1, V2 và V3; mức tăng lớn nhất thuộc V2, từ 2.968,80 lên 3.626,74 token mỗi câu. V4 là ngoại lệ trong mẫu này, với token và độ trễ attacked thấp hơn clean. Các giá trị độ trễ có thể chịu ảnh hưởng của cache, trạng thái provider và routing động, nên được xem là thống kê mô tả của một lần chạy chứ không phải đặc tính cố định của hệ thống.

8.7 So sánh với kết quả trước khi tấn công

Kết quả trước đây là benchmark clean trên toàn bộ 1.273 câu; kết quả mới là benchmark paired trên 50 câu đầu. Bảng dưới đây dùng kết quả clean cũ làm mốc tham chiếu và trình bày thêm clean/attacked mới. Chênh lệch giữa cột “clean cũ” và “clean mới” không được diễn giải là tác động của tấn công, vì kích thước mẫu và phạm vi benchmark khác nhau.

Bảng 7: Đối chiếu benchmark sạch cũ với thí nghiệm prompt injection mới.

Variant	Cũ clean / 1273	Cũ Acc.	Mới clean / 50	Mới attacked / 50	ASR
V0	1140	89.76%	86.0%	68.0%	24.0%
V1	1152	90.71%	84.0%	72.0%	20.0%
V2	1165	91.73%	86.0%	68.0%	20.0%
V3	1184	93.23%	86.0%	68.0%	20.0%
V4	1172	92.28%	86.0%	74.0%	18.0%

Trong phạm vi paired 50 câu, V3 giảm từ 86,0% clean xuống 68,0% khi bị chèn payload, tức giảm 18,0 điểm phần trăm; 10,0% số câu chuyển từ đúng sang sai ngoài các trường hợp model vốn đã sai hoặc thay đổi theo hướng khác mục tiêu. Kết quả này bổ sung một chiều đánh giá an toàn cho bảng benchmark cũ: accuracy cao trên câu hỏi sạch không đồng nghĩa với khả năng kháng prompt injection.

8.8 Hạn chế của thí nghiệm tấn công

Thí nghiệm mới chỉ sử dụng 50 câu đầu, một model (`deepseek-chat`), một chiến lược chính (`combine`) và một payload mục tiêu. Vì vậy, các tỷ lệ ASR và accuracy drop chưa thể đại diện cho toàn bộ 1.273 câu, mọi chiến lược tấn công hoặc mọi provider. Các nghiên cứu tiếp theo nên mở rộng lên toàn bộ test set, chạy riêng từng chiến lược, sử dụng nhiều seed/provider và bổ sung kiểm định bootstrap cho chênh lệch clean-attacked.

9 CONCLUSION

Nghiên cứu đã xây dựng và thực nghiệm khung Multi-Agent LLM `medqa-multiagent` trên toàn bộ 1.273 câu của tập kiểm thử MedQA-USMLE. Hệ thống cung cấp năm biến thể V0-V4 nhằm khảo sát tác động của truy hồi tri thức, phân tách vai trò Agent, Long-term Memory và Verifier. Chỉ mục RAG được xây dựng từ 18 nguồn textbooks, với tổng cộng 248.783 passages, và được truy vấn bằng hybrid FAISS + BM25 retrieval.

Kết quả thực nghiệm cho thấy V3 (Full System) đạt độ chính xác cao nhất với 1.184 câu trả lời đúng, tương ứng accuracy 93,23%. V0 đạt 89,76%, V1 đạt 90,71%, V2 đạt 91,73% và V4 đạt 92,28%. So với baseline V0, V3 cải thiện 3,46 điểm phần trăm về độ chính xác. Khoảng tin cậy bootstrap 95% của mức chênh lệch V0-V3 là $[+2,28; +4,65]$ điểm phần trăm và kiểm định McNemar cho $p < 0,001$. Khoảng tin cậy không chứa giá trị 0 cho thấy mức cải thiện của V3 so với V0 có ý nghĩa thống kê.

Các kết quả so sánh theo từng giai đoạn cũng cho thấy xu hướng cải thiện nhất quán. V1 cải thiện 0,94 điểm phần trăm so với V0 ($p = 0,0210$); V2 cải thiện 1,02 điểm phần trăm so với V1 ($p = 0,0175$); và V3 cải thiện 1,50 điểm phần trăm so với V2 ($p = 0,0018$). Điều này cho thấy các thành phần được bổ sung qua từng biến thể đã đóng góp tích cực vào hiệu quả dự đoán tổng thể trong cấu hình thực nghiệm hiện tại.

So sánh ablation giữa V3 và V4 cho thấy V3 có Verifier đạt accuracy 93,23%, trong khi V4 không có Verifier đạt 92,28%. Mức chênh lệch là 0,94 điểm phần trăm, với khoảng tin cậy bootstrap 95% là $[+0,22; +1,67]$ và kiểm định McNemar cho $p = 0,0118$. Phân tích theo từng câu ghi nhận 28 trường hợp V3 trả lời đúng trong khi V4 trả lời sai và 16 trường hợp theo chiều ngược lại; 1.226 trường hợp còn lại là hòa. Kết quả này cung cấp bằng chứng thống kê cho thấy Verifier có đóng góp tích cực, dù mức cải thiện tuyệt đối còn tương đối nhỏ.

Về hiệu quả tài nguyên, tổng chi phí API ước tính của V3 trên toàn bộ tập kiểm thử là \$3,3530, cao gấp khoảng 12,13 lần V0 (\$0,2764). Độ trễ trung bình của V3 là 4,12 giây/câu, cao gấp 2 lần V0 với 2,06 giây/câu. Như vậy, V3 cải thiện accuracy thêm 3,46 điểm phần trăm nhưng đồng thời làm tăng chi phí khoảng \$3,0766 và tăng độ trễ trung bình thêm 2,06 giây cho mỗi câu hỏi. Kết quả phản ánh sự đánh đổi rõ rệt giữa chất lượng dự đoán và hiệu quả sử dụng tài nguyên.

Thí nghiệm prompt injection bổ sung trên 50 câu đầu của official test set cho thấy hệ thống vẫn bị ảnh hưởng đáng kể khi câu hỏi bị chen chỉ dẫn độc hại. Đối với V3, clean accuracy là 86,0% nhưng attacked accuracy giảm còn 68,0%, tương ứng accuracy drop 18,0 điểm phần trăm, ASR 20,0% và prediction flip rate 22,0%. Trên cùng mẫu, V0 có ASR 24,0%, V1 20,0%, V2 20,0% và V4 18,0%. Như vậy, các thành phần RAG, Multi-Agent, Memory và Verifier cải thiện chất lượng trong điều kiện sạch nhưng chưa đủ để ngăn chặn prompt injection có chủ đích.

Kết quả tấn công phải được diễn giải cùng với giới hạn thiết kế: benchmark cũ sử dụng 1.273 câu clean, còn thí nghiệm mới sử dụng 50 câu paired clean/attacked. Do đó, bảng đối chiếu trong báo cáo có giá trị mô tả và kiểm tra độ bền ban đầu, chưa phải là kết luận thống kê cho toàn bộ test set. Một đánh giá hoàn chỉnh tiếp theo cần chạy toàn bộ 1.273 câu cho từng chiến lược tấn công và báo cáo khoảng tin cậy của accuracy drop, ASR và prediction flip rate.

Nhìn chung, kết quả thực nghiệm cho thấy việc kết hợp RAG, kiến trúc Multi-Agent, Long-term Memory và Verifier có thể cải thiện đáng kể chất lượng dự đoán trên MedQA-USMLE khi các thành phần được phối hợp phù hợp. Tuy nhiên, mức cải thiện đi kèm với chi phí API và thời gian xử lý lớn hơn. Các hướng phát triển tiếp theo cần tập trung tối ưu luồng truyền `memory_brief` vào Reasoner, nâng cao chất lượng truy hồi bằng hybrid retrieval, giảm các quyết định override không chính xác của Verifier và tối ưu số lượng lời gọi LLM. Ngoài ra, hệ thống cần được đánh giá trên nhiều model, nhiều seed và nhiều lần chạy độc lập để kiểm chứng tính ổn định cũng như khả năng khái quát của các kết quả hiện tại.

TÀI LIỆU THAM KHẢO

1. D. Jin, E. Pan, N. Oufettoho, X. S. Weng, H. Fang, and P. Szolovits, “What Disease does this Patient Have? A Large-scale Open Domain Question Answering Dataset from Medical Exams,” *Applied Sciences*, vol. 11, no. 14, p. 6421, 2021.
2. P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
3. Q. Wu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
4. Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 1831–1847, 2024. Code: <https://github.com/liu00222/Open-Prompt-Injection>.
5. S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “StruQ: Defending Against Prompt Injection with Structured Queries,” in *34th USENIX Security Symposium (USENIX Security 25)*, pp. 2383–2400, USENIX Association, 2025. Code: <https://github.com/Sizhe-Chen/StruQ>.
6. S. Chen, A. Zharmagambetov, S. Mahloutjifar, K. Chaudhuri, D. Wagner, and C. Guo, “SecAlign: Defending Against Prompt Injection with Preference Optimization,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, pp. 2833–2847, ACM, 2025. doi: <https://doi.org/10.1145/3719027.3744836>. Code: <https://github.com/facebookresearch/SecAlign>.